LABSHEET: 6

NAME:MANSHI VERMA

CUID: CU24250193

COMPUTER VISION LAB

ROLL NO:30

Name: Manshi verma CUID: CU24250193

1. Build a threshold based segmentation tool with adjustable global and adaptive thresholds.

```
!pip install -q opencv-python ipywidgets

import cv2
import matplotlib.pyplot as plt
import ipywidgets as w
from google.colab import files
from IPython.display import display, clear_output

# Upload image
f = files.upload()
img = cv2.imread(next(iter(f)))
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Controls
method = w.Dropdown(
    options=["Global", "Adaptive Mean", "Adaptive Gaussian"],
    description="Method:"
)
threshold = w.IntSlider(128, 0, 255, description="Threshold:")
block = w.IntSlider(11, 3, 51, 2, description="Block:")
C = w.IntSlider(2, -20, 20, description="C:")

out = w.Output()

def segment(x=None):
    with out:
        clear_output(wait=True)

        if method.value == "Global":
            _, result = cv2.threshold(
                gray, threshold.value, 255, cv2.THRESH_BINARY
            )

        elif method.value == "Adaptive Mean":
            result = cv2.adaptiveThreshold(
                gray, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
                cv2.THRESH_BINARY, block.value, C.value
            )

        else:
            result = cv2.adaptiveThreshold(
                gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                cv2.THRESH_BINARY, block.value, C.value
            )

        plt.figure(figsize=(10,4))
        plt.subplot(1,2,1)
        plt.imshow(gray, cmap="gray")
        plt.title("Original")
        plt.axis("off")

        plt.subplot(1,2,2)
        plt.imshow(result, cmap="gray")
        plt.title("Segmented")
        plt.axis("off")
        plt.show()

for x in [method, threshold, block, C]:
    x.observe(segment, names="value")

display(method, threshold, block, C, out)
segment()
```



```
4.9/4.9 MB 46.3 MB/s eta 0:00:00
```

Choose Files images (9).jpg

images (9).jpg(image/jpeg) - 48582 bytes, last modified: 9/28/2026 - 100% done

```
Saving images (9).jpg to images (9).jpg
```

Method: Global

Threshold: 128

Block: 13

C: 2

Original



Segmented



Name: Manshi verma CUID:CU24250193

2. Develop a region growing segmentation tool that segments an image starting from user selected seed points.

```
!pip install -q opencv-python matplotlib

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload image
uploaded = files.upload()
filename = list(uploaded.keys())[0]

img = cv2.imread(filename)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)

# Show image
plt.figure(figsize=(8, 5))
plt.imshow(img)
plt.title("Image - Select Seed Coordinates")
plt.axis("off")
plt.show()

# Enter seed point
x = int(input("Enter X coordinate: "))
y = int(input("Enter Y coordinate: "))

# Threshold
threshold = 15

# Region growing
visited = np.zeros(gray.shape, dtype=bool)
region = np.zeros(gray.shape, dtype=np.uint8)

seed_value = int(gray[y, x])
stack = [(y, x)]

while stack:
    cy, cx = stack.pop()

    if visited[cy, cx]:
        continue

    visited[cy, cx] = True

    if abs(int(gray[cy, cx]) - seed_value) <= threshold:
        region[cy, cx] = 255

        for dy, dx in [(-1,0), (1,0), (0,-1), (0,1)]:
            ny, nx = cy + dy, cx + dx

            if 0 <= ny < gray.shape[0] and 0 <= nx < gray.shape[1]:
                if not visited[ny, nx]:
                    stack.append((ny, nx))

# Display result
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(img)
plt.scatter(x, y, c="red", s=60)
plt.title("Selected Seed Point")
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
plt.imshow(region, cmap="gray")
plt.title("Region Growing Segmentation")
plt.axis("off")

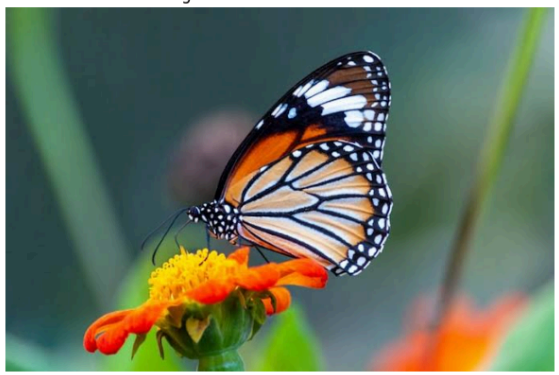
plt.show()
```

Choose Files images (2).jpg

images (2).jpg(image/jpeg) - 28138 bytes, last modified: 9/21/2026 - 100% done

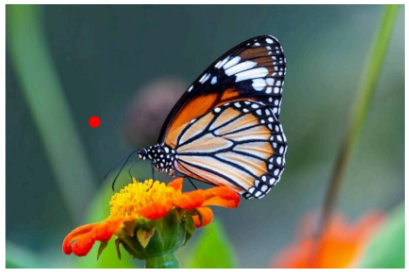
```
Saving images (2).jpg to images (2).jpg
```

Image - Select Seed Coordinates



```
Enter X coordinate: 150
Enter Y coordinate: 200
```

Selected Seed Point



Region Growing Segmentation



Name: Manshi verma CUID:CU24250193

3. Create a spilt and merge segmentation demo on a textured or satellite image.

```
!pip install -q opencv-python matplotlib

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload image
f = files.upload()
img = cv2.imread(next(iter(f)))
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Split & Merge
def split_merge(img, min_size=16, threshold=20):
    result = np.zeros_like(img)

    def process(x, y, w, h):
        roi = img[y:y+h, x:x+w]
        if roi.size == 0:
            return

        if np.std(roi) > threshold and w > min_size and h > min_size:
            w2, h2 = w//2, h//2
            process(x, y, w2, h2)
            process(x+w2, y, w-w2, h2)
            process(x, y+h2, w2, h-h2)
            process(x+w2, y+h2, w-w2, h-h2)
        else:
            result[y:y+h, x:x+w] = np.mean(roi)

    process(0, 0, img.shape[1], img.shape[0])
    return result.astype(np.uint8)

segmented = split_merge(gray)
```



```
# Display
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(gray, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(segmented, cmap="gray")
plt.title("Split & Merge Segmentation")
plt.axis("off")

plt.show()
```

Choose Files images (8).jpg

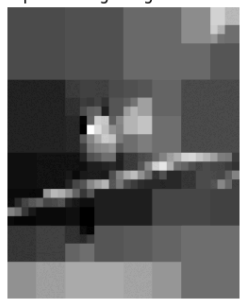
images (8).jpg(image/jpeg) - 16055 bytes, last modified: 9/28/2026 - 100% done

```
Saving images (8).jpg to images (8).jpg
```

Original



Split & Merge Segmentation



Name: Manshi verma CUID:CU24250193

4. Build a watershed segmentation tool to separate touching or overlapping objects(e.g. coins or cells)

```
!pip install -q opencv-python matplotlib

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload image
f = files.upload()
img = cv2.imread(next(iter(f)))
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Binary image
_, binary = cv2.threshold(
    gray, 0, 255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Noise removal
kernel = np.ones((3,3), np.uint8)
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel, iterations=2)

# Sure background
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# Distance transform
dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5)

# Sure foreground
_, sure_fg = cv2.threshold(
    dist, 0.4 * dist.max(), 255, 0
)
sure_fg = np.uint8(sure_fg)

# Unknown region
unknown = cv2.subtract(
    sure_bg, sure_fg
)

# Markers
_, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

# Watershed
img_color = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
markers = cv2.watershed(img_color, markers)

# Mark boundaries
img_color[markers == -1] = [255, 0, 0]

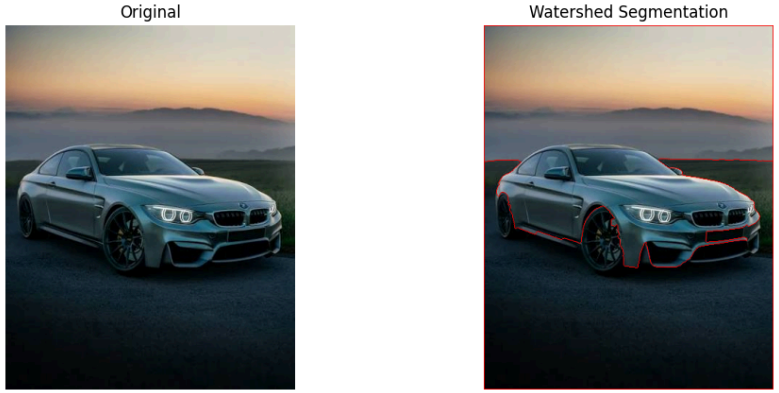
# Display
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")
```

```
plt.subplot(1,2,2)
plt.imshow(img_color)
plt.title("Watershed Segmentation")
plt.axis("off")

plt.show()
```

```
Choose Files images (10).jpg
images (10).jpg(image/jpeg) - 17561 bytes, last modified: 9/28/2026 - 100% done
Saving images (10).jpg to images (10).jpg
```



Name: Manshi verma CUID:CU24250193

5. Develop a k-Means or mean shift colour segmentation tool that clusters image regions by colour similarity.

```
!pip install -q opencv-python matplotlib

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload image
f = files.upload()
img = cv2.imread(next(iter(f)))
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Reshape image
pixels = img.reshape((-1, 3)).astype(np.float32)

# K-Means
K = 4
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 20, 1.0)

_, labels, centers = cv2.kmeans(
    pixels, K, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS
)

# Create segmented image
centers = np.uint8(centers)
segmented = centers[labels.flatten()]
segmented = segmented.reshape(img.shape)

# Display
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(img)
plt.title("Original")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(segmented)
plt.title(f"K-Means Segmentation (K={K})")
plt.axis("off")

plt.show()
```



```
Choose Files download (1).jpg
download (1).jpg(image/jpeg) - 6668 bytes, last modified: 9/28/2026 - 100% done
Saving download (1).jpg to download (1) (1).jpg
```



Name: Manshi verma CUID:CU24250193

6. Create an interactive foreground extraction tool using a graph based method such as Grabcut.

```
!pip install -q opencv-python matplotlib

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload image
f = files.upload()
img = cv2.imread(next(iter(f)))
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Rectangle around foreground
h, w = img.shape[:2]
rect = (10, 10, w-20, h-20)

# GrabCut
mask = np.zeros((h, w), np.uint8)
bgd = np.zeros((1, 65), np.float64)
fgd = np.zeros((1, 65), np.float64)

cv2.grabCut(
    img, mask, rect, bgd, fgd,
    5, cv2.GC_INIT_WITH_RECT
)

# Extract foreground
mask2 = np.where(
    (mask == 2) | (mask == 0), 0, 1
).astype("uint8")

result = img * mask2[:, :, np.newaxis]

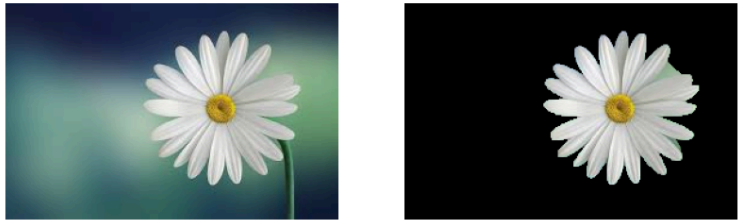
# Display
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(img)
plt.title("Original")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(result)
plt.title("GrabCut Foreground")
plt.axis("off")

plt.show()
```

```
Choose Files images (5).jpg
images (5).jpg(image/jpeg) - 4939 bytes, last modified: 9/21/2026 - 100% done
Saving images (5).jpg to images (5) (2).jpg
```



Name: Manshi verma CUID:CU24250193

7. Build a normalized cut based segmentation demo that partitions an image into meaningful regions.

```
!pip install -q scikit-image

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from skimage.segmentation import slic
from skimage import graph

# Upload image
f = files.upload()
img = cv2.cvtColor(cv2.imread(next(iter(f))), cv2.COLOR_BGR2RGB)

# Superpixels
segments = slic(img, n_segments=80, compactness=10, start_label=1)

# Region Adjacency Graph
g = graph.rag_mean_color(img, segments, mode='similarity')

# Normalized Cut
result = graph.cut_normalized(segments, g)

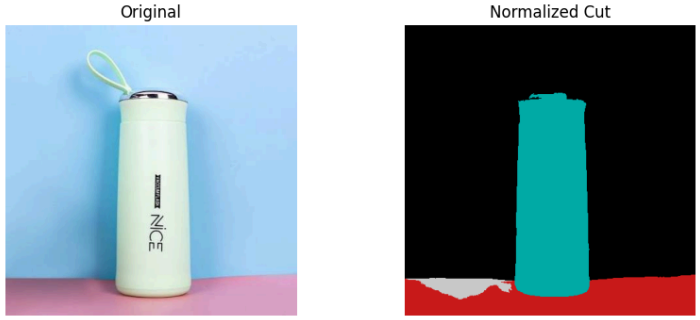
# Show result
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(img)
plt.title("Original")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(result, cmap="nipy_spectral")
plt.title("Normalized Cut")
plt.axis("off")
plt.show()
```

Choose Files images (12).jpg

images (12).jpg(image/jpeg) - 8824 bytes, last modified: 9/28/2026 - 100% done

```
Saving images (12).jpg to images (12).jpg
```



NAME: MANSHI VERMA

CUID:CU24250193

8. Develop a medical image segmentation tool (e.g. isolating a tumour or oragan outline from a grayscale scan)

```
!pip install -q opencv-python matplotlib

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload medical scan
f = files.upload()
img = cv2.imread(next(iter(f)), 0)

# Smooth + threshold
blur = cv2.GaussianBlur(img, (5,5), 0)
_, mask = cv2.threshold(blur, 0, 255,
                        cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Remove small noise
kernel = np.ones((3,3), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

# Display
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(img, cmap="gray")
plt.title("Medical Scan")
plt.axis("off")

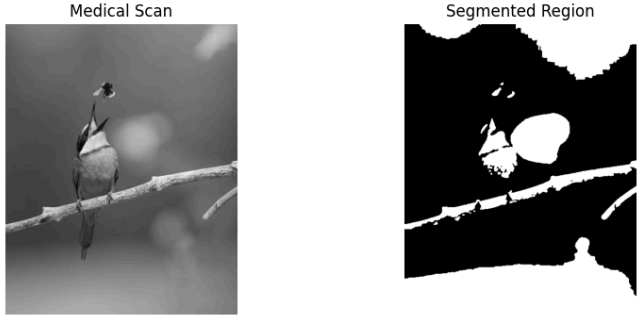
plt.subplot(1,2,2)
plt.imshow(mask, cmap="gray")
plt.title("Segmented Region")
plt.axis("off")

plt.show()
```


Choose Files images (8).jpg

images (8).jpg(image/jpeg) - 16055 bytes, last modified: 9/28/2026 - 100% done

```
Saving images (8).jpg to images (8) (1).jpg
```



NAME: MANSHI VERMA

CUID:CU24250193

9. Create a license plate or text region segmentation tool that isolates text areas from vehicles or document images.

```
!pip install -q opencv-python matplotlib

import cv2
import matplotlib.pyplot as plt
from google.colab import files

f = files.upload()
img = cv2.imread(next(iter(f)))

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.GaussianBlur(gray, (5,5), 0)

# Text/plate segmentation
_, mask = cv2.threshold(
    gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# Remove noise
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3,3))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

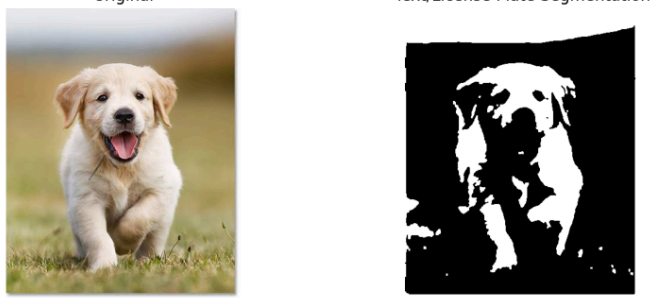
plt.subplot(1,2,2)
plt.imshow(mask, cmap="gray")
plt.title("Text/License Plate Segmentation")
plt.axis("off")

plt.show()
```

Choose Files images (3).jpg

images (3).jpg(image/jpeg) - 22691 bytes, last modified: 9/21/2026 - 100% done

```
Saving images (3).jpg to images (3).jpg
```



NAME: MANSHI VERMA

CUID:CU24250193

10. Build a evaluation dashboard that compares segmentation accuracy (Iou/Dice score) of different methods against ground truth masks.

```
!pip install -q opencv-python matplotlib

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload image
f = files.upload()
img = cv2.imread(next(iter(f)), 0)

# Ground truth (threshold)
_, gt = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

# Method 1: Otsu
_, m1 = cv2.threshold(img, 0, 255,
                      cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Method 2: Adaptive Threshold
m2 = cv2.adaptiveThreshold(
    img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, 11, 2)

def score(a, b):
    a, b = a > 127, b > 127
    inter = np.sum(a & b)
    iou = inter / np.sum(a | b)
    dice = 2 * inter / (np.sum(a) + np.sum(b))
    return iou, dice

s1 = score(gt, m1)
s2 = score(gt, m2)

print("Otsu      → IoU:", round(s1[0],3), "Dice:", round(s1[1],3))
print("Adaptive  → IoU:", round(s2[0],3), "Dice:", round(s2[1],3))

# Dashboard
plt.bar(["Otsu IoU","Otsu Dice","Adaptive IoU","Adaptive Dice"],
        [s1[0],s1[1],s2[0],s2[1]])
plt.ylim(0,1)
plt.ylabel("Score")
plt.title("Segmentation Evaluation Dashboard")
plt.show()
```

```
Choose Files download (1).jpg
download (1).jpg(image/jpeg) - 6668 bytes, last modified: 9/28/2026 - 100% done
Saving download (1).jpg to download (1) (2).jpg
Otsu      → IoU: 0.944 Dice: 0.971
Adaptive  → IoU: 0.483 Dice: 0.652
```

